

Pico Universal Programmer & Debugger

A 3-in-1 open-source hardware programmer for Raspberry Pi Pico (RP2040) with physical mode switching, status LEDs, and turnkey Arduino IDE integration.

CMSIS-DAP v2

Black Magic Probe

PicoRVD (WCH 1-Wire)

CH32V003

STM32

RP2040

ESP32

nRF52

SAMD

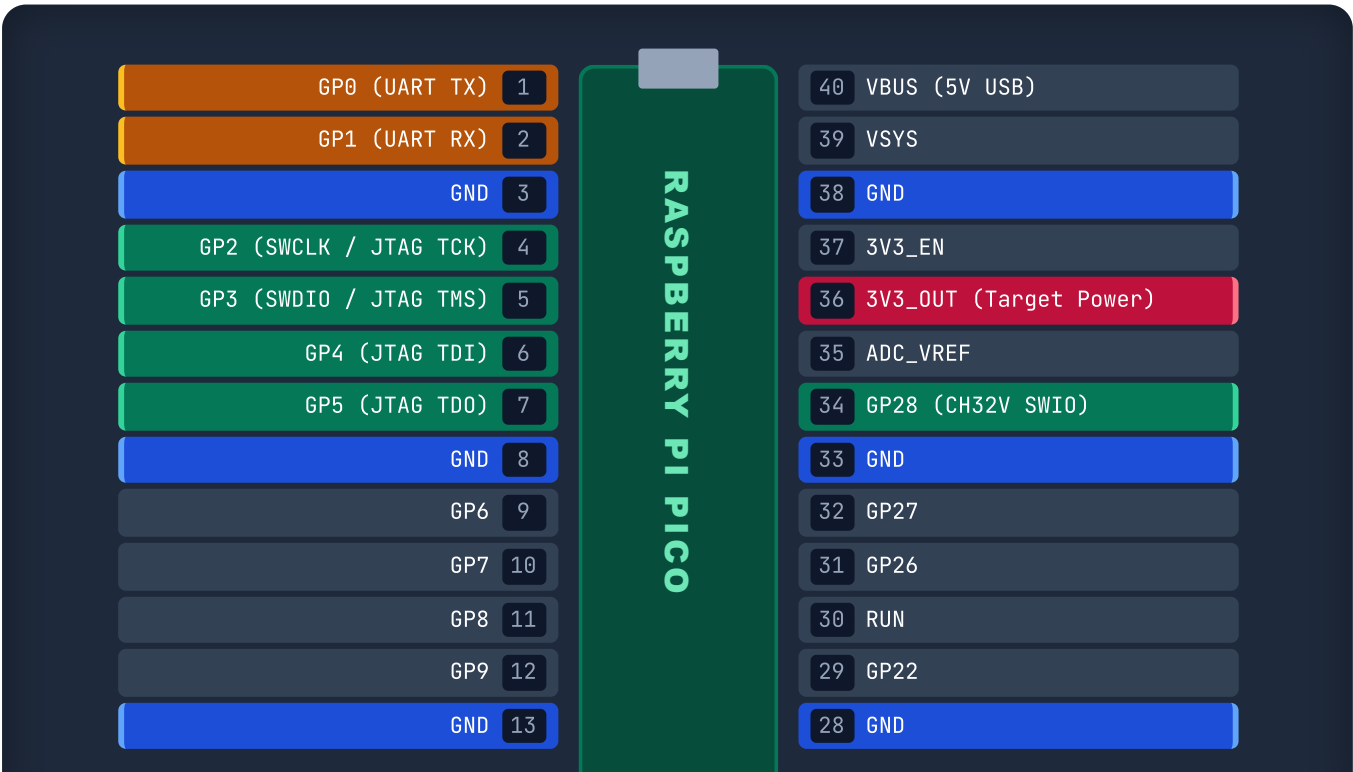
User Manual — Version 1.1 — September 2026

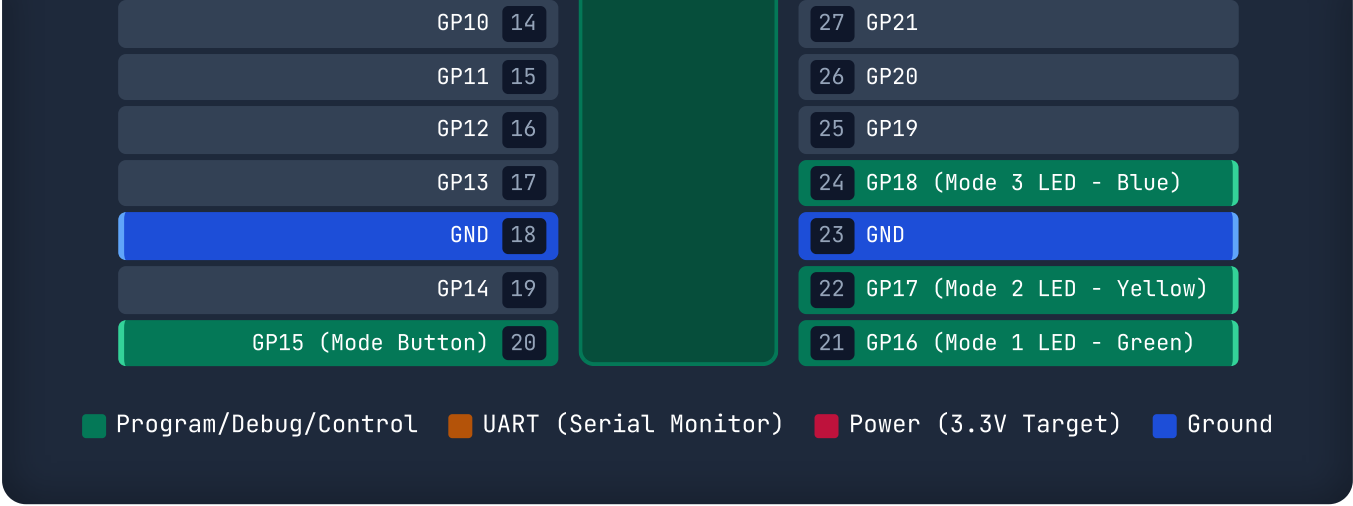
Table of Contents

#	Section
1	Master Pinout & Breadboard Wiring
2	Mode Operation & Button Controls
3	Target 1: WCH CH32V003 (1-Wire SWIO)
4	Target 2: STM32 Blue Pill / Black Pill (SWD)
5	Target 3: Raspberry Pi Pico / RP2040 (SWD)
6	Target 4: ESP32 / ESP32-S3 (4-Wire JTAG)
7	Serial Monitor (UART) Debugging
8	Arduino IDE One-Click Setup
9	Quick Reference Cheat Sheet
10	Electrical Safety Rules

1. Master Pinout & Breadboard Wiring

The Raspberry Pi Pico programmer relies on specific GPIO pins for SWD, JTAG, 1-Wire, UART, and controls. Wire your breadboard exactly according to the map below.





Control Assembly Instructions

Component	Pico Pin	Connection
Mode Button	Pin 20 (GP15)	Connect Button between GP15 and GND (Pin 18). Internal pull-up is enabled.
Green LED (Mode 1)	Pin 21 (GP16)	GP16 → LED Anode → LED Cathode → 330Ω Resistor → GND
Yellow LED (Mode 2)	Pin 22 (GP17)	GP17 → LED Anode → LED Cathode → 330Ω Resistor → GND
Blue LED (Mode 3)	Pin 24 (GP18)	GP18 → LED Anode → LED Cathode → 330Ω Resistor → GND

TIP

The three discrete LEDs and resistors are optional. The Pico's onboard green LED (GP25) also blinks 1, 2, or 3 times to indicate the active mode if you prefer not to build the full breadboard circuit.

2. Mode Operation & Button Controls

Mode	Indicator	Firmware Engine	Target Board Families
Mode 1	Green LED / 1 Blink	Raspberry Pi Debug Probe (CMSIS-DAP v2)	STM32, RP2040, ESP32, nRF52, SAMD, WCH V203/V307
Mode 2	Yellow LED / 2 Blinks	Black Magic Probe (Built-in GDB Server)	ARM Cortex-M, RISC-V (zero external software needed)
Mode 3	Blue LED / 3 Blinks	PicoRVD (WCH 1-Wire SWIO)	WCH CH32V003 (proprietary single-wire debug on PD1)

Button Controls

Action	Gesture	Feedback
Cycle Mode	Short press GP15 (< 1 second)	Advances to next mode: Mode 1 → Mode 2 → Mode 3 → Mode 1
Save Default Power-On Mode	Long press GP15 (> 2 seconds)	All LEDs flash rapidly 5 times — current mode is permanently saved to Flash memory as the default

3. Target 1: WCH CH32V003 (1-Wire SWIO)

The CH32V003 uses WCH's proprietary single-wire debug interface on pin **PD1**. This is the only way to flash a blank CH32V003 (it has no BOOT0 pin and no built-in UART bootloader).

Wiring Table

Pico Pin	Pico Function	CH32V003 Target	Notes
Pin 34	GP28	PD1 (SWIO)	Single-wire data line
—	1kΩ Resistor	Between 3.3V and PD1	Mandatory pull-up resistor
Pin 36	3V3 OUT	VDD (3.3V)	Powers the target chip
Pin 38	GND	GND	Common ground

⚠ CRITICAL: PULL-UP RESISTOR REQUIRED

The **1kΩ pull-up resistor** between 3.3V and the PD1 pin is absolutely mandatory. Without it, the 1-wire protocol cannot pull the signal high fast enough and programming will fail.

Arduino IDE Steps

- 1 Tap the Mode Button (GP15) until the ● Blue LED is on (or the onboard LED blinks 3 times).
- 2 In Arduino IDE: **Tools > Board > CH32V EVT Boards** → CH32V003 EVT
- 3 **Tools > Port** → Select your Pico's COM port
- 4 **Tools > Programmer** → Pico Universal: PicoRVD (CH32V003 1-Wire SWIO)
- 5 Press Ctrl + Shift + U (Upload Using Programmer)

4. Target 2: STM32 Blue Pill / Black Pill (SWD)

ARM Cortex-M chips use the standard 2-wire Serial Wire Debug (SWD) protocol. This includes STM32F103 (Blue Pill), STM32F401/F411 (Black Pill), STM32G0, and all Nucleo boards.

Wiring Table

Pico Pin	Pico Function	STM32 Target Pin	Notes
Pin 4	GP2	SWCLK (Clock)	SWD Clock
Pin 5	GP3	SWDIO (Data)	SWD Data
Pin 36	3V3 OUT	3.3V	Power target from Pico
Pin 8	GND	GND	Common ground
Pin 1	GP0 (UART TX)	RX (e.g. PA10)	Serial Monitor: Pico sends to target
Pin 2	GP1 (UART RX)	TX (e.g. PA9)	Serial Monitor: Pico listens to target

Arduino IDE Steps

- 1

Set the Pico to

● Mode 1

 (Green LED / 1 Blink).
- 2

Tools > Board > STM32 boards groups → Select your board (e.g.

Generic STM32F1 series

)
- 3

Tools > Upload method →

OpenOCD
- 4

Tools > Programmer →

Pico Universal: CMSIS-DAP (SWD)
- 5

Press

Ctrl

 +

Shift

 +

U

5. Target 3: Raspberry Pi Pico / RP2040 (SWD)

Use your programmer Pico to flash and debug *another* Raspberry Pi Pico via the 3-pin SWD debug header on the target board.

Wiring Table

Programmer Pico Pin	Function	Target Pico Pin
Pin 4 (GP2)	SWCLK	Target SWCLK (Debug Header)
Pin 5 (GP3)	SWDIO	Target SWDIO (Debug Header)
Pin 8 (GND)	GND	Target GND (Debug Header)
Pin 1 (GP0)	UART TX (Pico sends)	Target GP1 (UART0 RX)
Pin 2 (GP1)	UART RX (Pico listens)	Target GP0 (UART0 TX)

Arduino IDE Steps

- 1

Set the Pico to

● Mode 1

 (Green LED / 1 Blink).
- 2

Tools > Board > Raspberry Pi RP2040 Boards → Raspberry Pi Pico
- 3

Tools > Programmer → Pico Universal: CMSIS-DAP (Picoprobe)
- 4

Press

Ctrl

 +

Shift

 +

U

6. Target 4: ESP32 / ESP32-S3 (4-Wire JTAG)

Standard ESP32 microcontrollers support JTAG debugging via 4 dedicated pins.

Wiring Table

Programmer Pico Pin	JTAG Signal	Standard ESP32	ESP32-S3
Pin 4 (GP2)	TCK (Clock)	GPIO 13	GPIO 39
Pin 5 (GP3)	TMS (Mode)	GPIO 14	GPIO 42
Pin 6 (GP4)	TDI (Data In)	GPIO 12	GPIO 41

Pin 7 (GP5)	TDO (Data Out)	GPIO 15	GPIO 40
Pin 8 (GND)	GND	GND	GND

Arduino IDE Steps

1 Set the Pico to **Mode 1** (Green LED / 1 Blink).

2 **Tools > Programmer** → CMSIS-DAP

3 Press **Ctrl** + **Shift** + **U**

7. Serial Monitor (UART) Debugging

The Pico programmer provides a built-in **USB-to-UART serial bridge** over the same USB cable, so you can view your target board's `Serial.println()` output without an additional FTDI adapter.

Pico Programmer Pin	Function	Target Pin
Pin 1 (GP0)	UART TX (Pico sends data)	Target RX
Pin 2 (GP1)	UART RX (Pico receives data)	Target TX
Pin 8 (GND)	GND (Required)	Target GND

In Arduino IDE, open **Tools > Serial Monitor** (`Ctrl` + `Shift` + `M`) to see live output from your target board in real time.

8. Arduino IDE One-Click Setup

Step 1: Install Board Packages

In Arduino IDE, go to **File > Preferences** and add these URLs to the **Additional Boards Manager URLs** field (comma-separated):

Board Family	URL
WCH CH32V	https://github.com/openwch/board_manager_files/raw/main/package_ch32v_index.json
STM32	https://github.com/stm32duino/BoardManagerFiles/raw/main/package_stmicroelectronics_index.json
RP2040	https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json
ESP32	https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

Then go to **Tools > Board > Boards Manager** and install the cores you need.

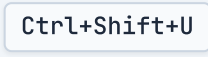
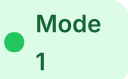
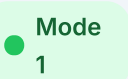
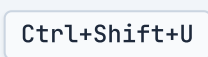
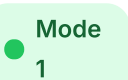
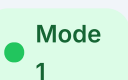
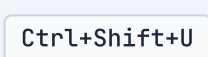
Step 2: Run the Installer Script

Double-click the installer script located in your project folder:

```
arduino\install_arduino_support.bat
```

It auto-detects your installed Arduino cores and installs the custom programmer recipes. **Restart Arduino IDE** after running.

9. Quick Reference Cheat Sheet

Target Board	Pico Mode	Tools > Programmer	Wiring Pins	Upload
CH32V003	 Mode 3	PicoRVD	GP28 → PD1 (+1kΩ)	
CH32V203/307	 Mode 1	CMSIS-DAP	GP2/GP3 → SWCLK/SWDIO	
STM32	 Mode 1	CMSIS-DAP (SWD)	GP2/GP3 → SWCLK/SWDIO	
RP2040	 Mode 1	CMSIS-DAP (Picolprobe)	GP2/GP3 → SWCLK/SWDIO	
ESP32 (JTAG)	 Mode 1	CMSIS-DAP	GP2/3/4/5 → TCK/TMS/TDI/TDO	

10. Electrical Safety Rules

⚠ VOLTAGE WARNING (3.3V LOGIC)

The Raspberry Pi Pico GPIO pins are strictly **3.3V** and are **NOT 5V tolerant**. If your target board runs at 5V (e.g., Arduino Uno/Nano), you **must** use a bidirectional logic level shifter on all signal lines. Feeding 5V into the Pico will permanently destroy the RP2040.

⚠ COMMON GROUND

Always connect **GND** between the Pico and your target board, even if both boards are powered by separate USB ports. Without a common ground reference, communication will be unreliable and you risk hardware damage.

⚠ CH32V003 PULL-UP RESISTOR

Never omit the **1kΩ pull-up resistor** on the CH32V003 PD1 line. Without it, the single-wire protocol cannot operate and the chip will not respond.

